ZOMATHON - 2026

Problem Statement 1

Adaptive Multi-Signal Trust Framework for Accurate Kitchen Prep Time (KPT) Prediction

Team : The Zomatechs

Members : Sanya Arya, Manas Dhamija

(For clarity refer : Google Drive Video Link)

Problem Understanding & Context

Zomato's Kitchen Prep Time (KPT) prediction plays a critical role in delivery accuracy, rider efficiency, and customer satisfaction. However, the current system relies heavily on merchant-marked "Food Order Ready" (FOR) events, which often do not accurately reflect actual food preparation completion time.

This leads to inconsistent and noisy input signals for KPT prediction.

Key Limitations Identified

1. Rider-Influenced FOR Marking

Merchants sometimes mark food as "ready" when the rider arrives, rather than when preparation is actually complete. This creates artificially inflated prep-time data.

2. Lack of Visibility into Kitchen Rush

The system only observes Zomato orders and does not capture total kitchen workload (including dine-in and competitor orders). This limits real-time rush estimation.

3. Human & Operational Bias

Manual marking behavior varies across merchants and time periods. Some merchants consistently mark early, others mark late, and behavior changes during peak hours.

Downstream Impact of Inaccurate KPT

Noisy and biased KPT signals result in:

- Increased rider waiting time
- Early or late rider arrivals
- ETA fluctuations (P50/P90 error increase)
- Higher order delays and cancellations
- Increased rider idle time

Thus, the core issue is not model capability — it is signal reliability. We shift the problem from prediction optimization to signal trust optimization.

Proposed Solution

Enhancing KPT Prediction through Adaptive Synchronization

The primary objective of this proposed solution is to dynamically adjust the Kitchen Prep Time (KPT) to accurately reflect real-time kitchen rush and reduce rider waiting times. Current KPT predictions heavily rely on manual merchant inputs, which are prone to human error and manipulation.

To solve this, our framework evaluates the exact discrepancy between the rider's physical arrival and the actual food preparation time. The foundational metric of this system is the **Rider Waiting Time (W)**, which acts as a proxy for live kitchen rush.

Variable Definitions:

- **T1**: The exact timestamp the rider marks "arrived and waiting" at the restaurant geo-fence.
- **P**: The timestamp the food is marked as "Prepared" or dispatched.
- **Formula**: **W** = P – T1

Initial Assumptions for Baseline: The food is marked as prepared exactly when it is ready, and the rider triggers the arrival status precisely upon reaching the location. Under these conditions, W is directly proportional to the restaurant's live rush, allowing the system to continuously measure and correct KPT dynamically.

Mathematical Model: Adaptive Rider Synchronization

To actively correct the KPT, the system utilizes an adaptive learning formula governed by a specific error function. This acts as a feedback loop.

Variables Defined:

- $\boldsymbol{KPT_0}$**:** The current Kitchen Prep Time predicted by the system.
- $\boldsymbol{KPT_{new}}$**:** The adjusted Kitchen Prep Time for future orders.
- **α(Alpha)**: An adjustment factor acting as a learning rate or sensitivity dial (typically set between 0.1 and 0.5 depending on the desired aggressiveness).

Core Adjustment Formula:

$$KPT_{new} = KPT_0 + \alpha \times W$$

Practical Examples of the Adaptive System

- **Case A: Rider Arrives Early (High Wait Time, W > 0)**
  - ***Scenario:*** Rider arrives before the food is ready, indicating kitchen delay.
  - ***Calculation:*** If P = 7:30 and T1 = 7:25, then W = +5 minutes.
  - ***Action***: $KPT_{new} = KPT_0 + \alpha(5)$.
  - ***Result:*** The KPT increases, assigning the next rider later.
- **Case B: Rider Arrives Late (Negative Wait Time, W < 0)**
  - ***Calculation:*** If P = 7:25 and T1 = 7:30, then W = -5 minutes.
  - ***Action***: $KPT_{new} = KPT_0 - \alpha(5)$.
  - ***Result***: The KPT decreases, assigning the next rider earlier.
- **Case C: Perfect Timing (W approx 0)**
  - ***Result***: No change is made. The system is stabilized.

Smoothing Function to Prevent Volatility:

To avoid sudden, erratic jumps in estimations due to a single outlier order, a moving average smoothing function is applied using a weight factor B:

$$KPT_{final} = (1 - \beta) \times KPT_{old} + (\beta \times KPT_{new})$$

Advanced Control Systems and Edge Case Mitigation

Zero wait time is theoretically perfect but practically risky. Ideally, a rider should arrive a few minutes before the food is fully prepared to ensure an immediate handoff and preserve food quality.

Step 1: Define the Target Buffer

- Let B equal the Target Buffer Time (e.g., 3 to 5 minutes).
- The ideal waiting time (W_{target}) should equal B.
- *Example*: If the food is ready at 7:30, the rider should arrive between 7:25 and 7:27.

Step 2: Evaluate Current Waiting

- If $W > B$, the rider is too early (wasting rider time).
- If $W < B$, the rider is too late (food is getting cold).

Step 3: Define the Error Term (E) This term measures the exact deviation from the desired buffer.

$$\boxed{Error(E) = W - B}$$

Final Control Formula:

$$\boxed{KPT_{new} = KPT_0 + (\alpha \times E)}$$

The "Dead Zone Band" for Maximum Stability
Instead of a single fixed value for B, the system defines a buffer range to prevent the algorithm from over-correcting minor fluctuations.

- $B_{min} = 3$ minutes
- $B_{max} = 5$ minutes

Control Logic Flow:

- If $W < B_{min} \rightarrow$ Decrease KPT.
- If $W > B_{max} \rightarrow$ Increase KPT.
- If $3 \leq W \leq 5 \rightarrow$ Make no change.
- *Business Impact*: This creates a highly stable, realistic operational band that prevents algorithmic jitter.

Simulation & Quantitative Validation

To validate the effectiveness of our proposed Adaptive KPT Correction Framework, we developed a Python-based simulation model using Google Colab. The simulation recreates real-world order flow conditions by generating variations in kitchen preparation time, rider arrival time, waiting time deviations, and adaptive KPT updates using our proposed control formulas (error-based correction + smoothing mechanism).

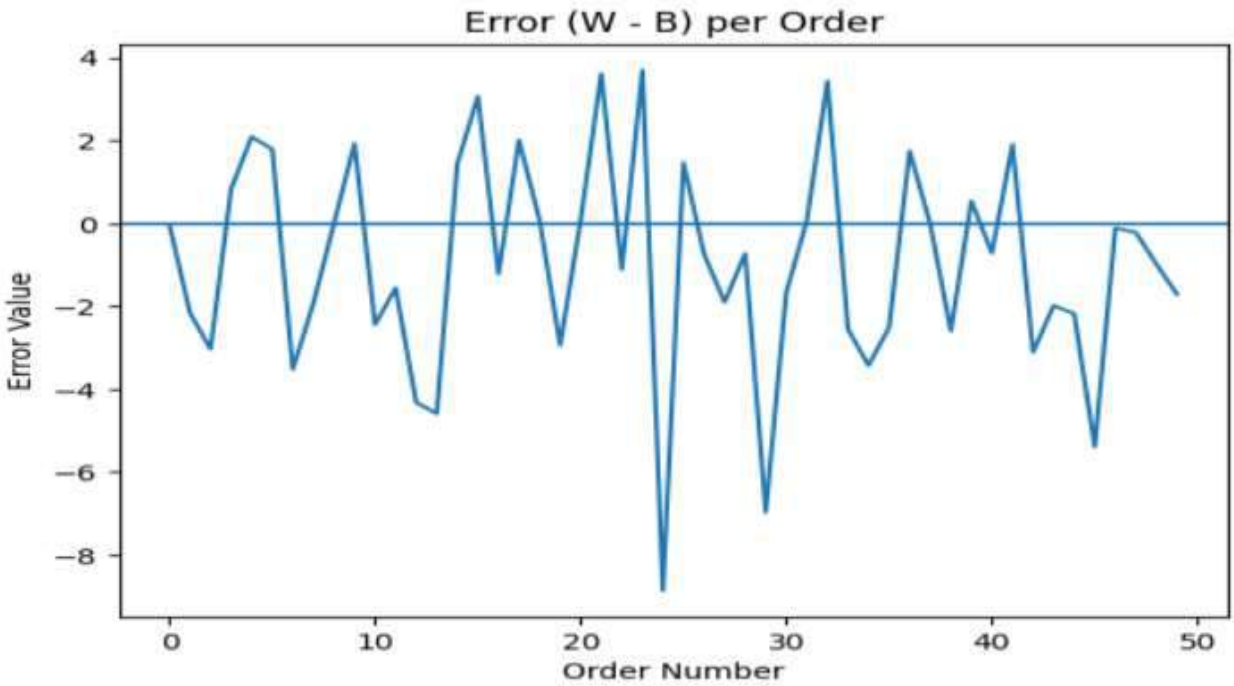
This quantitative model demonstrates how rider waiting signals can dynamically correct KPT predictions, reduce systematic bias, and gradually stabilize dispatch timing around the desired 3–5 minute buffer window.

The following graphs illustrate:

- Rider Waiting Time fluctuations
- Error (deviation from target buffer)
- Progressive KPT stabilization over multiple orders

[Google Collab Simulation Link]

This graph shows how rider waiting time fluctuates around the 4-minute target buffer, helping the system identify when the kitchen is under rush (above line) or when riders are arriving late (below line) for adaptive KPT correction.



This graph shows the deviation of actual rider waiting time from the 4-minute target buffer, where positive errors indicate riders arriving too early (kitchen rush) and negative errors indicate late rider arrival, guiding adaptive KPT adjustments.

This graph demonstrates how the adaptive feedback mechanism progressively stabilizes and optimizes the predicted KPT over multiple orders based on rider waiting-time corrections.

Decoding Human Operational Biases

The core of our mathematical framework relies on a critical foundational assumption: that merchants input data (specifically, marking "Food Prepared") accurately and in real-time. However, in a fast-paced, high-stress kitchen environment, human operational biases inevitably disrupt this ideal flow.

Merchants may manipulate these inputs—sometimes unintentionally due to the chaos of a busy kitchen, but often intentionally to artificially inflate their performance metrics on the platform. When these manual inputs become unreliable, our baseline algorithmic assumptions fail, which would traditionally lead to severely skewed Kitchen Prep Time (KPT) predictions. To maintain strict data integrity, our algorithm is built to dynamically recognize, categorize, and mathematically correct these human-driven anomalies on the fly.

We classify these operational biases into two primary edge cases: **LPEM** and **EPLM**.

CASE 1: Late Prepared Early Marked (LPEM)

The Scenario: This occurs when a merchant marks "Food Prepared" significantly earlier than the food is actually ready. This is typically a false alarm triggered deliberately to make the restaurant's preparation metrics appear much faster than reality.

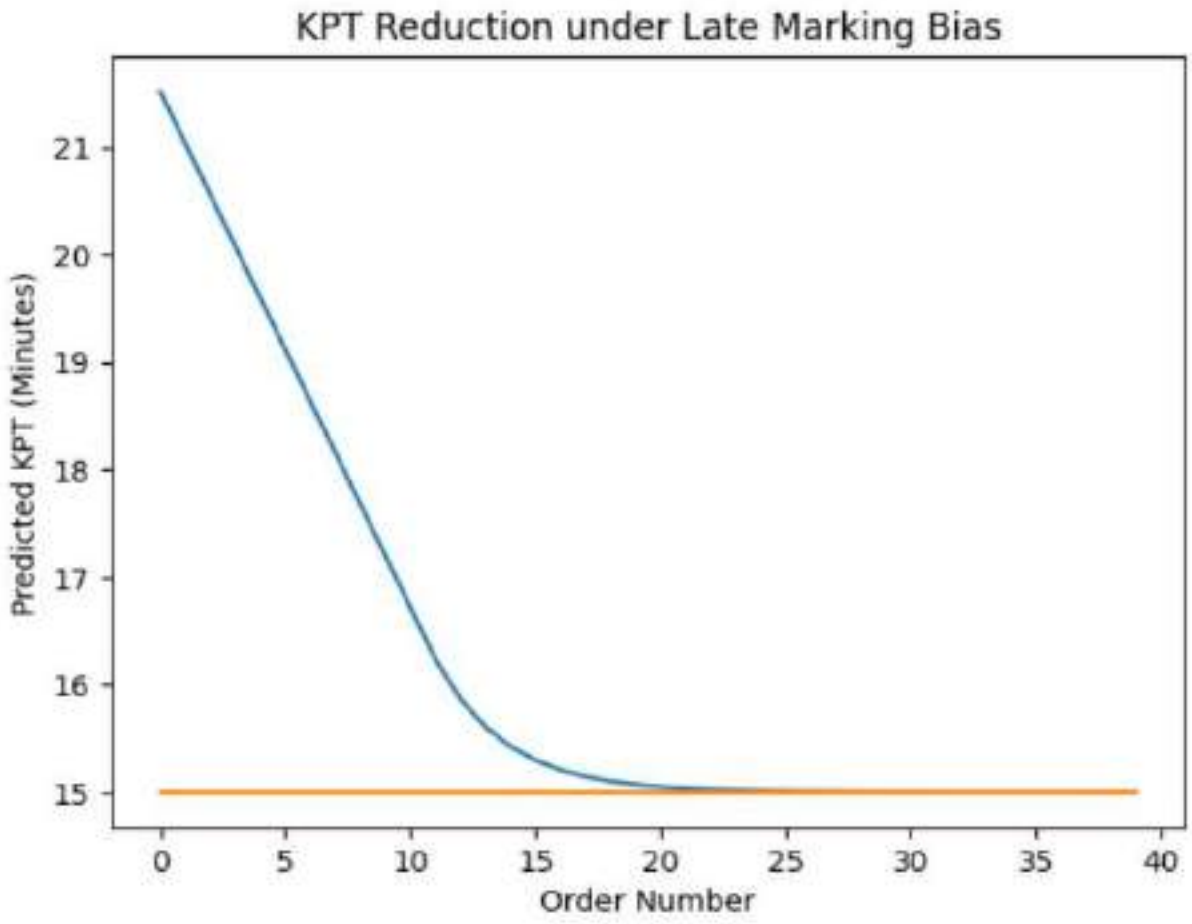
Detection and Correction Mechanism:

- To tackle this bias, the system introduces a secondary validation check: the time difference (T2) between the actual *Food Dispatched* time (when the rider physically leaves) and the Rider's *Arrival Time*.
- ***Ideal Scenario*:** If the food was genuinely prepared when marked, the order should be dispatched the moment the rider arrives at the counter.
- ***LPEM Scenario*:** Because the food is not actually ready (a false alarm), the waiting time between the rider's physical arrival and the actual dispatch will be suspiciously longer than usual.
- ***Resolution*:** Upon detecting this anomaly, the system actively discards the merchant’s manual "Food Prepared" timestamp. Instead, the algorithm falls back on the actual *Food Dispatched* time as the true variable (P) and uses this to calculate KPT confidence levels.

- ***Penalty***: If a merchant continuously exhibits this behaviour, automated warnings are issued, and their manual inputs are temporarily deprioritized by the algorithm.

CASE 2: Early Prepared Late Marked (EPLM)

The Scenario: This is the opposite operational bias. The food was genuinely prepared on time (or early), but the merchant forgot or actively delayed marking "Food Prepared" on their device—often waiting until the exact moment the rider walks through the door to update the system.



The Solution: This graph illustrates that even under the Early Prepared–Late Marked (EPLM) condition, where merchants delay marking food as prepared, **our adaptive KPT mechanism intelligently reduces the predicted KPT until it reaches the true kitchen preparation limit,** thereby effectively correcting the bias without causing underestimation.

Merchant Incentive & Behavior Improvement Layer:

While mathematical corrections treat the symptoms of bad data, behavioural modelling treats the root cause. To reduce EPLM and encourage real-time logging, we introduce a gamified incentive-based system.

Restaurants are categorized based on how their manually marked prep times compare against actual observed delivery timelines:

- 🟢 **Usually On-Time**: High trust score; manual inputs are trusted.
- 🟡 **Usually Early**: Medium trust score.
- 🔴 **Usually Late (High EPLM/LPEM offenders)**: Low trust score; manual inputs are heavily scrutinized.

Goal: This transparent classification aims to instil self-discipline among restaurant staff, incentivizing them to log updates accurately at the correct time to maintain a "Green" status and ensure better platform visibility, rather than delaying the update.

The Chain Effect and Complete Behaviour Loop :

There is a crucial psychological connection between these two cases. Introducing incentives improves general marking behaviour and solves EPLM, but it may simultaneously create a new operational risk:

➡️ *In an attempt to secure the coveted "Green" rating, some restaurants may try to game the system by marking food as prepared earlier than reality.*

This directly leads right back to the **Late Prepared Early Marked (LPEM)** situation. However, this risk is safely neutralized because our framework functions as a fully closed, self-regulating system.

The Complete Closed-Loop System:

1. **Incentivize:** The categorization system encourages accurate, real-time marking and naturally eliminates EPLM.
2. **Detect:** Any resulting early or late manipulation (like the resulting LPEM bias) is automatically detected via rider arrival signals and waiting time analysis.
3. **Correct:** The confidence-based LPEM correction logic overrides the bad data, preventing it from corrupting the KPT algorithm.
4. **Improve:** Facing a system they cannot game, merchant behavior gradually aligns with reality over time.

Conclusion

The accuracy of Kitchen Prep Time (KPT) prediction is fundamentally dependent on the reliability of operational input signals. In real-world environments, merchant marking inconsistencies, rider-influenced behavior, and unobserved kitchen rush introduce significant noise into these signals, leading to rider inefficiencies and ETA instability.

Our proposed Adaptive Multi-Signal Trust Framework addresses this challenge by strengthening the signal layer rather than modifying the core prediction model.
By combining:

- Rider arrival validation
- Confidence-weighted signal correction
- Adaptive synchronization formulas
- Merchant behavioral classification and incentives

the framework transforms noisy operational events into validated, high-confidence prep-time inputs.
Importantly, the system functions as a closed-loop mechanism:

Detection → Correction → Prevention

This ensures that inaccurate signals are not only corrected in real time but that long-term data quality continuously improves through behavioral alignment.
The proposed approach is scalable, requires no additional hardware, integrates seamlessly with existing workflows, and directly impacts key success metrics including rider waiting time, ETA prediction error (P50/P90), order delays, and rider idle time.

By shifting the focus from model optimization to signal trust optimization, the framework provides a practical, sustainable, and production-ready pathway toward more reliable KPT prediction and improved delivery efficiency at scale.